Scheduling and Travel Graph

Revision: `4405d167fc323f33be76e2c7f6614fd21cc574a8` (`main`). This volume teaches a deterministic service scenario and a topology-weighted shortest-path estimate. The code does not dispatch real trains or receive live passenger counts.

1. Where these pieces fit

`app/scheduling_optimizer.py` loads two inputs when Python imports the module: the operations CSV (`data/cleaned/metro_line_operations.csv`) and, if it exists, the estimated interchange share JSON (`data/synthetic/line_distribution_model.json`). `routes.api_predict` sends forecast demand and selected date/hour facts to `optimize_station_schedule`. The optimizer returns line-level dictionaries for the JSON response. The operations rows are configurable project assumptions; their labels and comments do not establish that values are official or verified.

`app/network_topology.py` provides ordered station sequences and transfer definitions. `app/travel_graph.py` maps the topology names to canonical names in the station CSV, creates one graph node per `(station, line)`, connects consecutive stations, and adds weighted transfer edges. `routes.py` builds the graph at import and queries it for a selected origin. Both are estimates: edges use coordinate-derived straight-line distance divided by a fixed average speed, or a fixed fallback, rather than actual timetable and track geometry.

2. Scheduling: understand the units first

The optimizer compares **passenger demand per hour at the selected station** with a modeled **boarding clearance budget per arriving train**. It is not measuring all passengers already onboard, and its utilization denominator is not nominal train capacity multiplied by trains per hour. For a line, the core relationship is:

```
clearance budget per train = comfort capacity * (1 - assumed through-load fraction)
```

```
clearance offered per hour = trains per hour * clearance budget per train
```

```
utilization = station line demand / clearance offered per hour * 100
```

The through-load fractions (65% at peak, 40% off-peak) are hard-coded scenario assumptions. Thus the output should be described as a model-based station boarding capacity scenario, not actual occupancy. It does not simulate alighting, onboard loads between stations, passenger queues, train-by-train movement, or a live dispatch control system.

3. `is_peak_hour`

Signature: `is_peak_hour(hour: int, day_of_week: int, is_workday=None) -> bool` (`app/scheduling_optimizer.py:38-49`). `day_of_week` follows Python's weekday convention: Monday 0 through Sunday 6. If `is_workday` is explicitly supplied it overrides that weekday inference. A false workday immediately returns `False`; otherwise hours 8, 9, 10 and 17, 18, 19, 20 return true. The code uses whole integer hours, so 20:59 is represented as hour 20 and is peak; it does not model the half-hour cutoff in its docstring.

Example: `is_peak_hour(9, 2, True)` is true; `is_peak_hour(9, 2, False)` is false. The second result demonstrates the override. Invalid types are not validated; comparisons may raise `TypeError`.

Interview: Why accept both weekday and `is_workday`? The optional override lets the caller use the project's calendar/holiday classification rather than assuming every weekday is a workday. What is the limitation? The peak windows are coarse hour bins and do not encode exact minutes.

4. Line split helper and headway rounding

`get_interchange_line_split(station: str, all_lines: list) -> dict` (`52-64`) returns saved proportions when the station is a key in the loaded interchange model. Otherwise it evenly divides 1 by the count and rounds that share to four decimal places. Empty `all_lines` returns `{}`. It does not normalize a supplied JSON split or validate that its keys match `all_lines` or sum to one. This function affects how one station total becomes per-line modeled demand.

`round_to_practical_headway(headway_min, min_headway, max_headway) -> float` (`67-78`) first clamps a proposed headway into the physical/policy interval. It filters the `STANDARD_HEADWAYS` list to values inside those bounds and selects the valid value with smallest absolute distance from the clamped target. If no standard interval fits, it returns the clamped value rounded to two decimals. A tie resolves to the first item in list order because Python's `min` keeps the first equal key. The bounds are not independently validated; if minimum exceeds maximum, `max(min, min(target, max))` resolves to minimum.

Example: target 4.2 min, allowed 2.1-8.0 min. Valid choices include 4.0 and 4.5. Distances are 0.2 and 0.3, so result is 4.0. Smaller headway means more frequent trains: `60 / 4.0 = 15.0 trains/hour`.

5. `optimize_line_schedule` worked trace

Signature (81-82): `optimize_line_schedule(line, station_name, hour, day_of_week, line_demand, is_workday=None) -> dict`. The route passes an integer line-demand estimate and date context. A normal return is a dictionary consumed by the station optimizer and ultimately rendered to JSON/UI. The exact complete implementation is reproduced in Appendix A.

Step-by-step calculation (illustrative, using the tracked Yellow Line assumptions)

Assume weekday 09:00, demand 1,000 riders/hour, and the Yellow Line CSV row: comfort capacity 1,920; nominal capacity 2,400; minimum signaling headway 2.1 minutes; maximum policy headway 8 minutes; peak baseline headway 2.3 minutes; fleet 64; turnaround 150 minutes.

1. Fleet limit: `floor(64 * 60 / 150) = floor(25.6) = 25 trains/hour`. 2. Signaling limit: `floor(60 / 2.1) = 28 trains/hour`; absolute maximum is `min(25, 28) = 25`. 3. Policy minimum frequency: `ceil(60 / 8) = 8 trains/hour`. 4. Peak baseline: 2.3 min, so existing frequency is `round(60 / 2.3, 1) = 26.1 trains/hour`. 5. Peak through-load is assumed 0.65. Clearance per train is `int(1,920 * (1 - .65)) = 672` riders. 6. Existing offered boarding clearance is `int(26.1 * 672) = 17,539` riders/hour. Demand 1,000 does not exceed this. Since 1,000 is not less than 45% of 17,539? 45% is about 7,893, yes it is less, but 09:00 is not in the energy-saving-hour allowlist, so it maintains baseline. 7. Frequency is then clamped to the fleet max 25, making raw headway `60/25 = 2.4`. The nearest allowed standard headway is 2.3 (distance .1) rather than 2.5 (.1 tie? Both are .1; list order selects 2.3), so recommendation becomes 2.3 and 26.1 trains/hour. **This reveals a constraint mismatch:** frequency is clamped before conversion to headway, then practical rounding can raise frequency above the clamp. For this data, the recommendation reports 26.1 while the calculated fleet maximum is 25. Treat this as a potential bug/risk, not a guarantee of feasible fleet use. 8. The returned utilization is demand divided by final clearance: `1000 / int(26.1*672) * 100`, about 5.7%. `trains_deployed_rec = min(64, ceil(150/2.3)) = 64`; the capped fleet count conceals that the unbounded cycle/headway requirement is 66 sets.

The arithmetic demonstrates why returned fields should be interpreted carefully: headway rounding and `trains_deployed_rec` currently do not guarantee that all reported values satisfy every configured constraint. The CSV and code do not prove the assumed capacities/fleet/headways are official.

Control flow, edge cases, and risks

• An unknown line tries a case-insensitive substring match; if none match, it silently falls back to the first CSV index row. Empty CSV index would fail at `_OPS_DF.index[0]`.

• Demand-driven frequency is `line_demand / clearance_budget_per_train`. The `max(..., 250)` clearance floor prevents division by zero but means low-capacity assumptions could be masked.

• Action thresholds compare demand with current modeled clearance, and decrease service only at selected off-peak hours. `status_action` is chosen before frequency constraints, so action may say increase when the constrained recommendation cannot meet demand.

• Negative demand is not rejected. It can lower target frequency, while utilization becomes negative; this produces nonsensical output rather than an input error.

• Zero or invalid turnaround/headway values in CSV can cause division by zero or nonsensical bounds. Startup CSV loading only confirms parsing, not semantic validity.

• The function uses `nominal_cap` for total line capacity, but uses comfort capacity minus through-load for boarding clearance. These answer different questions.

• At 105% and 88% comparisons, strict > means exactly 105% is HIGH, exactly 88% is MODERATE, and exactly 65% is LOW.

• Wait time uses the textbook random-arrival approximation `headway / 2`; it assumes arrivals are uniform and ignores bunching and passenger schedules.

Time complexity is constant per line call apart from the small fixed headway list scan. Memory use is constant for the returned summary, excluding module-level pandas/JSON data.

6. `optimize_station_schedule`

Signature (252-254): `optimize_station_schedule(station, primary_line, all_lines, hour, day_of_week, total_footfall, is_workday=None, service_open=True) -> dict`.

If more than one line is listed, it calls the split helper. Otherwise it maps the full demand to `primary_line`. It loops over `all_lines`, rounds each share times station total, and calls the line optimizer. When `service_open` is false, it overwrites operating metrics with zero/`None`, sets `SERVICE_CLOSED`, and explains no trains operate. The final primary recommendation is actually the line with largest `line_demand`, not necessarily the supplied `primary_line`.

Risks: empty `all_lines` yields an empty recommendations list, then `max(...)` raises `ValueError`. Missing share keys use an equal split fallback, but the JSON split may sum to something other than 1, so rounded line demands need not add to station total. When closed, each line optimizer is first called with zero demand, then the resulting metrics are overwritten, doing unnecessary work. `total_station_footfall` is set to zero on closure. No board/alight or onboard state is modeled.

7. Network topology as data

`app/network_topology.py` is declarative data, not executable routing logic. Each `LINES[line]` list is an ordered sequence. The graph builder connects each adjacent pair and does not connect every possible pair on a line. `INTERCHANGES` lists station names and corresponding line keys; it is the explicit source for transfer edges. `INTERCHANGE_PENALTY_MIN = 5.0` applies to every listed transfer, regardless of the comment noting stations vary. `DEFAULT_SEGMENT_MIN = 2.5` is used when endpoint coordinates are absent/repeated or considered implausible.

`ALIASES` maps a normalized topology label to a canonical CSV spelling. Matching omissions mean stations/edges can disappear without a hard error. `INTERCHANGES` must agree with line membership and names; otherwise a graph node can represent a transfer line that has no adjacent track edge. The module docstring's source claims are not independently verified by the repository audit.

8. Graph construction details

`_normalize` removes a fixed set of footnote glyphs, strips/lowercases, normalizes dash variants, and removes commas/periods. It replaces only one occurrence of two consecutive spaces with one, rather than fully collapsing arbitrary whitespace. `_load_stations` reads CSV via `DictReader`, trims station and coordinate fields, converts non-empty coordinates to float, and indexes by normalized name; duplicate normalized names overwrite earlier rows.

`_is_placeholder_coord` returns true when a coordinate pair repeats for more than two station rows, or either coordinate is `None`. `_haversine_km` computes great-circle surface distance on a sphere with radius 6,371 km. The formula returns straight-line surface distance, not rail track length.

`_match_topology_to_canonical` builds a set of topology station names, tries normalized exact match, then the alias dictionary. Set iteration order is not guaranteed, but mapping results are key based so the final lookup is stable unless duplicate normalized names exist.

`build_graph(csv_path)` makes an adjacency dictionary. For each line, `zip(stops, stops[1:])` creates consecutive pairs. Missing name matches skip the edge. Valid coordinates yield `max((distance / 32 km/h) * 60, 1 minute)`; results above 8 minutes fall back to 2.5. Invalid/repeated coordinates also use 2.5. Every track edge is inserted in both directions, so the graph treats travel as symmetric. For each interchange, every pair among listed lines gets an edge weighted 5 minutes, also symmetric. The node tuple `(canonical_station, line)` preserves line identity; transfer cost is paid only when switching between such nodes.

Potential risks: coordinates are point-to-point rather than track distance; average speed is fixed; max-edge fallback can hide bad inputs; fallback values can distort routes; line/transfer names can mismatch; all transfers have one cost; `MAX_SEGMENT_MIN` and source geographic quality determine routing quality. Construction is linear in total listed station occurrences plus the small pairwise line count at each interchange; graph space is O(V+E), where V is station-line nodes and E is adjacency entries.

9. Dijkstra: `shortest_times_from`

Signature (`139-167`): `shortest_times_from(graph, station_name) -> dict[str, float]`. The start is multi-source: every graph node whose station matches starts at distance zero. This means a selected interchange starts on each of its lines with no transfer penalty, suitable for station-level travel-time estimates that begin at the station. A route that changes lines later pays the transfer edge weight.

The min-heap stores `(distance, node)`. Each loop pops the currently least-cost candidate; a `visited` set prevents repeated expansion. For each neighbor, candidate distance is `d + w`; if it improves the best-known distance, the code updates `dist` and pushes a fresh heap entry. Stale heap entries are naturally skipped after the node has been finalized. Finally, it collapses line-specific nodes to the minimum time per station and removes the origin.

Hand-worked graph

Illustrative graph: start A on line X. A-X to B-X costs 3 minutes; B-X to C-X costs 4; B-X transfer to B-Y costs 5; B-Y to D-Y costs 2. Starting at A, heap progression is A(0) -> B-X(3) -> B-Y(8) and C-X(7) -> D-Y(10). Therefore shortest times: B=3, C=7, D=10. Going to D pays the 5-minute transfer. Dijkstra requires nonnegative weights; the builder creates positive weights. If the requested station is absent, `start_nodes` is empty, so the result is `{}` without an explicit not-found error.

Complexity is `O((V + E) log V)` using a binary heap and adjacency lists; each successful relaxation pushes at most an edge-scale number of heap entries, conventionally summarized as O(E log V). Space is O(V+E) including adjacency, distance, heap, and visited structures.

10. Interview questions and exercises

1. **Why use `(station, line)` nodes?** So the graph can distinguish being at one physical station on two line platforms and charge a transfer cost when moving between line identities. 2. **What makes Dijkstra valid here?** Edge weights are nonnegative travel minutes. A negative transfer/travel edge would invalidate the greedy finalization rule. 3. **Is `capacity_utilization_pct` onboard train occupancy?** No. It compares a station/hour demand estimate against modeled per-hour boarding clearance derived from comfort capacity and assumed through-load. 4. **Does the scheduler dispatch trains?** No. It returns a rule-based scenario recommendation; there is no train state, control system, or real-time data connection. 5. **How would you test headway constraints?** Unit tests for bounded inputs, exact standard intervals, tie behavior, no valid standard entries, and invariants after rounding (`min <= h <= max`, `60/h <= fleet/signal limit`). The last invariant currently exposes the rounding issue.

Exercise: For Rajiv Chowk, use the JSON split, a total of 10,000 modeled riders/hour, and `all_lines=["Blue Line", "Yellow Line"]`. Calculate the rounded demand sent to each line. Then describe how the algorithm chooses `primary_recommendation`.

Solution: Blue: `round(10000 * .5154) = 5154`; Yellow: `round(10000 * .4846) = 4846`; they sum to 10,000. It runs a line recommendation for each and selects the dictionary with greatest `line_demand`, so Blue Line is primary in this example. This selection is code behavior, not a statement about operational importance.

11. File connections and verification

• Caller: `app/routes.py` calls station optimization and shortest-time routing when assembling a prediction response.

• Upstream demand: runtime forecasting returns total station demand; line split config distributes it across lines.

• Upstream network: cleaned station CSV supplies coordinates/names; topology supplies adjacency and interchanges.

• Downstream: route JSON goes to `dashboard.js`; the schedule is displayed as a scenario, and travel times are shown in the destination table.

• Proposed improvements: validate CSV bounds on load; preserve constraints after rounding; distinguish boarding flow from onboard occupancy in labels; add uncertainty ranges; source and version operations assumptions; test disconnected/unknown stations; base travel time on published segment schedules/track paths and station-specific transfers.

Appendix source blocks

The following appendix reproduces complete source files with original line numbers. It is intentionally consecutive: consult it alongside the explanations above to verify exact executable behavior.

Appendix A - complete source excerpts

These excerpts come from the documented commit. Original line numbers are printed at left. For print readability, Unicode dash variants are normalized to ASCII hyphens and other non-ASCII source symbols are shown as explicit Unicode escape notation; this does not change the source line numbers or code meaning. Long lines wrap with indented continuations.

Complete line and station scheduling optimizer module `app/scheduling_optimizer.py:1-306`

```python
"""
DMRC Realistic Train Frequency and Scheduling Optimization Engine.

Provides transit-grade operational headway and frequency optimization based on:
1. Predicted passenger demand over time.
2. Verified line train passenger capacity (nominal, comfort 80%, crush 120%).
3. Baseline scheduled train frequency and headways (DMRC official peak vs off-peak timetables).
4. Minimum safe train separation (CATC/CBTC signaling safety headway limit).
5. Maximum policy headway (service frequency upper bound).
6. Active line fleet availability and turnaround cycle times.
7. Passenger waiting times and platform clearance limits.
8. Explainable operational recommendations.
"""
import os
import json
import math
import pandas as pd

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OPS_PATH = os.path.join(BASE_DIR, 'data', 'cleaned', 'metro_line_operations.csv')
LINE_MODEL_PATH = os.path.join(BASE_DIR, 'data', 'synthetic', 'line_distribution_model.json')

# Load operational parameters and line models at module import
_OPS_DF = pd.read_csv(OPS_PATH).set_index('Line')
if os.path.exists(LINE_MODEL_PATH):
    with open(LINE_MODEL_PATH, 'r', encoding='utf-8') as f:
        _LINE_DIST_MODEL = json.load(f)
else:
    _LINE_DIST_MODEL = {"interchanges": {}}

# Practical headway dispatch intervals (minutes) common in metro operations
STANDARD_HEADWAYS = [
    2.0, 2.15, 2.3, 2.5, 2.75, 3.0, 3.25, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0,
    6.5, 7.0, 7.5, 8.0, 9.0, 10.0, 12.0, 15.0
]


def is_peak_hour(hour: int, day_of_week: int, is_workday=None) -> bool:
    """
    Returns True if the hour falls into Delhi Metro peak rush hours on a weekday.
    Weekday morning peak: 08:00 - 11:00 (hours 8, 9, 10)
    Weekday evening peak: 17:00 - 20:30 (hours 17, 18, 19, 20)
    Saturdays/Sundays are treated as off-peak.
    """
    if is_workday is None:
        is_workday = day_of_week < 5
    if not is_workday:
        return False
    return (8 <= hour <= 10) or (17 <= hour <= 20)


def get_interchange_line_split(station: str, all_lines: list) -> dict:
    """
    Returns {line: proportion} for an interchange station.
    """
    interchanges = _LINE_DIST_MODEL.get("interchanges", {})
    if station in interchanges:
        return interchanges[station].get("proportions", {})

    # Fallback equal distribution
    if all_lines:
        share = round(1.0 / len(all_lines), 4)
        return {l: share for l in all_lines}
    return {}


def round_to_practical_headway(headway_min: float, min_headway: float, max_headway: float) -> float:
    """
    Rounds a continuous optimal headway to the closest practical metro dispatch interval,
    strictly obeying signaling minimum and policy maximum bounds.
    """
    bounded = max(min_headway, min(headway_min, max_headway))
    valid_headways = [h for h in STANDARD_HEADWAYS if min_headway <= h <= max_headway]
    if not valid_headways:
        return round(bounded, 2)
    # Find closest standard interval
    closest = min(valid_headways, key=lambda x: abs(x - bounded))
    return round(closest, 2)


def optimize_line_schedule(line: str, station_name: str, hour: int, day_of_week: int,
                           line_demand: int, is_workday=None) -> dict:
    """
    Optimizes train scheduling for a specific line at a given station and hour.
    """
    if line not in _OPS_DF.index:
        # Default fallback if line key has slight naming variant
        line_match = [idx for idx in _OPS_DF.index if line.lower() in idx.lower()]
        line = line_match[0] if line_match else _OPS_DF.index[0]

    ops = _OPS_DF.loc[line]
    nominal_cap = int(ops['Nominal_Capacity'])
    comfort_cap = int(ops['Comfort_Capacity'])
    min_signal_headway = float(ops['Min_Signaling_Headway_Min'])
    max_policy_headway = float(ops['Max_Policy_Headway_Min'])
    fleet_size = int(ops['Active_Fleet_Size'])
    turnaround_min = float(ops['Turnaround_Time_Min'])

    # Maximum frequency physically supported by rolling stock fleet and turnaround time
    # Max trains per hour = Fleet / (Turnaround / 60)
    fleet_max_freq = math.floor((fleet_size * 60.0) / turnaround_min)
    signaling_max_freq = math.floor(60.0 / min_signal_headway)
    abs_max_freq = min(signaling_max_freq, fleet_max_freq)
    abs_min_freq = math.ceil(60.0 / max_policy_headway)

    peak = is_peak_hour(hour, day_of_week, is_workday)

    # Baseline scheduled headway from official DMRC operations
    if hour in [5, 6, 22, 23]:
        # Taper hours (early morning or late night)
        existing_headway = min(max_policy_headway, float(ops['Baseline_OffPeak_Headway_Min']) * 1.3)
    elif peak:
        existing_headway = float(ops['Baseline_Peak_Headway_Min'])
    else:
        existing_headway = float(ops['Baseline_OffPeak_Headway_Min'])

```

```
    existing_freq = round(60.0 / existing_headway, 1)

    # Transit Boarding Clearance Budget per Train:
    # During peak, trains arrive with ~65% through load, leaving 35% comfort capacity for boarding.
    # During off-peak, trains arrive with ~40% through load, leaving 60% comfort capacity for boarding.
    through_load_factor = 0.65 if peak else 0.40
    clearance_budget_per_train = max(int(comfort_cap * (1.0 - through_load_factor)), 250)

    # Existing offered clearance capacity for boarding at this station
    existing_clearance_capacity = int(existing_freq * clearance_budget_per_train)
    existing_line_capacity = int(existing_freq * nominal_cap)

    # Demand-driven frequency required to comfortably clear passenger boarding
    required_freq_for_demand = line_demand / clearance_budget_per_train

    # Baseline provides network connectivity across the entire line.
    # Recommended frequency must respect corridor baseline, but scale up if demand surges.
    if line_demand > existing_clearance_capacity:
        target_freq = required_freq_for_demand
        status_action = "INCREASE_FREQUENCY"
    elif line_demand < (existing_clearance_capacity * 0.45) and not peak and hour in [5, 6, 13, 14, 22, 23]:
        # Demand is light enough that frequency can be safely relaxed for energy efficiency
        target_freq = max(abs_min_freq, required_freq_for_demand * 1.1)
        status_action = "DECREASE_FREQUENCY"
    else:
        target_freq = existing_freq
        status_action = "MAINTAIN_SCHEDULE"

    # Clamping against physical signaling and fleet constraints
    constrained_freq = max(abs_min_freq, min(target_freq, abs_max_freq))
    raw_headway = 60.0 / constrained_freq

    # Round to realistic practical dispatch headway
    rec_headway = round_to_practical_headway(raw_headway, min_signal_headway, max_policy_headway)
    rec_freq = round(60.0 / rec_headway, 1)

    # Recomputed metrics with final recommended headway
    rec_clearance_capacity = int(rec_freq * clearance_budget_per_train)
    rec_total_line_capacity = int(rec_freq * nominal_cap)
    capacity_shortfall = rec_clearance_capacity - line_demand

    # Capacity Utilization (%)
    if rec_clearance_capacity > 0:
        utilization_pct = round((line_demand / rec_clearance_capacity) * 100.0, 1)
    else:
        utilization_pct = 0.0

    # Crowding & Platform Risk Assessment
    if utilization_pct > 105.0:
        crowding_risk = "CRITICAL_OVERCROWDING"
        crowding_badge = "Severe Crowd Alert"
        crowding_color = "#dc3545"  # Red
    elif utilization_pct > 88.0:
        crowding_risk = "HIGH_CROWDING"
        crowding_badge = "High Demand"
        crowding_color = "#fd7e14"  # Orange
    elif utilization_pct > 65.0:
        crowding_risk = "MODERATE"
        crowding_badge = "Optimal Load"
        crowding_color = "#28a745"  # Green
    else:
        crowding_risk = "LOW"
        crowding_badge = "Comfortable Capacity"
        crowding_color = "#17a2b8"  # Teal

    # Average Passenger Wait Time: E[W] = Headway / 2
    avg_wait_time_existing = round(existing_headway / 2.0, 1)
    avg_wait_time_rec = round(rec_headway / 2.0, 1)

    # Required active train sets deployed on corridor for this headway
    # Trains needed = Turnaround / Headway
    trains_deployed_rec = min(fleet_size, math.ceil(turnaround_min / rec_headway))

    # Construct transparent explainable narrative
    explanation_parts = []
    if status_action == "INCREASE_FREQUENCY":
        explanation_parts.append(
            f"Passenger demand on {line} at {station_name} ({line_demand:,} riders/hr) exceeds "
            f"baseline clearance capacity ({existing_clearance_capacity:,} riders/hr). "
            f"Recommended increasing service from {existing_freq:.1f} to {rec_freq:.1f} trains/hr "
            f"(headway reduced from {existing_headway:.1f}m to {rec_headway:.1f}m)."
        )
    elif status_action == "DECREASE_FREQUENCY":
        explanation_parts.append(
            f"Passenger demand is light ({line_demand:,} riders/hr). "
            f"Recommended moderating service from {existing_freq:.1f} to {rec_freq:.1f} trains/hr "
            f"(headway relaxed from {existing_headway:.1f}m to {rec_headway:.1f}m) to conserve energy "
            f"while safely respecting the {max_policy_headway:.1f}m policy maximum."
        )
    else:
        explanation_parts.append(
            f"Existing baseline timetable ({existing_freq:.1f} trains/hr, {existing_headway:.1f}m headway) "
            f"comfortably handles the projected demand ({line_demand:,} riders/hr) at {utilization_pct}% capacity utilization."
        )

    if rec_headway <= min_signal_headway + 0.05:
        explanation_parts.append(
            f"\u26a0\ufe0f Service frequency is operating at or near the absolute signaling safety limit ({min_signal_headway:.1f} min)."
        )
    if capacity_shortfall < 0:
        explanation_parts.append(
            f"\u26a0\ufe0f Capacity shortfall of {abs(capacity_shortfall):,} passengers/hr detected due to signaling throughput limit. "
            f"Station crowd management marshals recommended."
        )

    explanation = " ".join(explanation_parts)

    return {
        "line": line,
        "is_peak": peak,
        "line_demand": line_demand,
        "existing_headway_min": existing_headway,
        "existing_trains_per_hour": existing_freq,
        "existing_clearance_capacity": existing_clearance_capacity,
        "existing_line_capacity": existing_line_capacity,
        "recommended_headway_min": rec_headway,
        "recommended_trains_per_hour": rec_freq,
```

```
        "recommended_clearance_capacity": rec_clearance_capacity,
        "recommended_total_line_capacity": rec_total_line_capacity,
        "capacity_shortfall_surplus": capacity_shortfall,
        "capacity_utilization_pct": utilization_pct,
        "avg_wait_time_existing_min": avg_wait_time_existing,
        "avg_wait_time_rec_min": avg_wait_time_rec,
        "trains_deployed_rec": trains_deployed_rec,
        "fleet_size_available": fleet_size,
        "min_signaling_headway_min": min_signal_headway,
        "max_policy_headway_min": max_policy_headway,
        "crowding_risk": crowding_risk,
        "crowding_badge": crowding_badge,
        "crowding_color": crowding_color,
        "status_action": status_action,
        "explanation": explanation
    }


def optimize_station_schedule(station: str, primary_line: str, all_lines: list,
                              hour: int, day_of_week: int, total_footfall: int,
                              is_workday=None, service_open=True) -> dict:
    """
    Evaluates demand and scheduling for the entire station, including line-wise breakdowns
    for interchange stations.
    """
    is_interchange = len(all_lines) > 1
    line_splits = get_interchange_line_split(station, all_lines) if is_interchange else {primary_line: 1.0}

    line_recommendations = []
    for line in all_lines:
        share = line_splits.get(line, 1.0 / len(all_lines))
        line_demand = round(total_footfall * share)
        line_rec = optimize_line_schedule(
            line, station, hour, day_of_week, line_demand if service_open else 0, is_workday=is_workday
        )
        if not service_open:
            line_rec.update({
                'line_demand': 0,
                'existing_headway_min': None,
                'existing_trains_per_hour': 0,
                'existing_clearance_capacity': 0,
                'existing_line_capacity': 0,
                'recommended_headway_min': None,
                'recommended_trains_per_hour': 0,
                'recommended_clearance_capacity': 0,
                'recommended_total_line_capacity': 0,
                'capacity_shortfall_surplus': 0,
                'capacity_utilization_pct': 0,
                'avg_wait_time_existing_min': 0,
                'avg_wait_time_rec_min': 0,
                'trains_deployed_rec': 0,
                'crowding_risk': 'SERVICE_CLOSED',
                'crowding_badge': 'Metro closed',
                'crowding_color': '#64748b',
                'status_action': 'SERVICE_CLOSED',
                'explanation': 'Metro service is closed at the selected time. No trains are operating and modeled passenger demand
 is zero.'
            })
        line_rec["demand_share_pct"] = round(share * 100.0, 1)
        line_recommendations.append(line_rec)

    # Aggregate summary (using primary line or highest-demand line)
    primary_rec = max(line_recommendations, key=lambda r: r['line_demand'])

    return {
        "station": station,
        "is_interchange": is_interchange,
        "total_station_footfall": total_footfall if service_open else 0,
        "all_lines": all_lines,
        "primary_line": primary_rec["line"],
        "line_distribution_metadata": _LINE_DIST_MODEL.get("metadata", {}),
        "line_recommendations": line_recommendations,
        "primary_recommendation": primary_rec
    }
```

Complete hand-authored line topology and interchange data `app/network_topology.py:1-169`

```
"""
Real Delhi Metro network topology: station order along each line, and the
interchange (transfer) stations between lines.

Source: published DMRC route maps / line-by-line station orderings
(cross-checked against Wikipedia's per-line articles). This intentionally
matches the *stable* pre-2025 network, since that's what DELHI_METRO_DATA.csv
covers (no 2025/2026-opened stations appear in it).

This is the piece that was completely missing from the original project:
DELHI_METRO_DATA.csv only ever had (Station, Line, Latitude, Longitude) -- no
sequence, no interchange info, and every station appeared on exactly one
line. Without this file there was no way to know which stations are
*adjacent*, so the old code fell back to straight-line distance between
every possible pair.

Each line is a simple ordered list -- consecutive entries are the stations
you actually ride between, in order.
"""

LINES = {
    "Red Line": [
        "Shaheed Sthal", "Hindon", "Arthala", "Mohan Nagar", "Shyam Park",
        "Major Mohit Sharma", "Raj Bagh", "Shaheed Nagar", "Dilshad Garden",
        "Jhilmil", "Mansarovar Park", "Shahdara", "Welcome", "Seelampur",
        "Shastri Park", "Kashmere Gate", "Tis Hazari", "Pul Bangash",
        "Pratap Nagar", "Shastri Nagar", "Inderlok", "Kanhaiya Nagar",
        "Keshav Puram", "Netaji Subhash Place", "Kohat Enclave", "Pitam Pura",
        "Rohini East", "Rohini West", "Rithala",
    ],
    "Yellow Line": [
        "Samaypur Badli", "Rohini Sector 18", "Haiderpur", "Jahangirpuri",
        "Adarsh Nagar", "Azadpur", "Model Town", "GTB Nagar", "Vishwa Vidyalaya",
        "Vidhan Sabha", "Civil Lines", "Kashmere Gate", "Chandni Chowk",
        "Chawri Bazar", "New Delhi", "Rajiv Chowk", "Central Secretariat",
        "Udyog Bhawan", "Lok Kalyan Marg", "Jor Bagh", "INA", "AIIMS",
        "Green Park", "Hauz Khas", "Malviya Nagar", "Saket", "Qutab Minar",
        "Chhatarpur", "Sultanpur", "Ghitorni", "Arjan Garh", "Guru Dronacharya",
        "Sikandarpur", "MG Road", "IFFCO Chowk", "HUDA City Centre",
    ],
    "Blue Line": [
        "Dwarka Sector 21", "Dwarka Sector 8", "Dwarka Sector 9",
        "Dwarka Sector 10", "Dwarka Sector 11", "Dwarka Sector 12",
        "Dwarka Sector 13", "Dwarka Sector 14", "Dwarka", "Dwarka Mor",
        "Nawada", "Uttam Nagar West", "Uttam Nagar East", "Janakpuri West",
        "Janakpuri East", "Tilak Nagar", "Subhash Nagar", "Tagore Garden",
        "Rajouri Garden", "Ramesh Nagar", "Moti Nagar", "Kirti Nagar",
        "Shadipur", "Patel Nagar", "Rajendra Place", "Karol Bagh",
        "Jhandewalan", "Ramakrishna Ashram Marg", "Rajiv Chowk",
        "Barakhambha Road", "Mandi House", "Supreme Court", "Indraprastha",
        "Yamuna Bank", "Akshardham", "Mayur Vihar -I", "Mayur Vihar Extension",
        "New Ashok Nagar", "Noida Sector 15", "Noida Sector 16",
        "Noida Sector 18", "Botanical Garden", "Golf Course",
        "Noida City Centre", "Noida Sector 34", "Noida Sector 52",
        "Noida Sector 61", "Noida Sector 59", "Noida Sector 62",
        "Noida Electronic City",
    ],
    "Blue Line branch": [
        "Yamuna Bank", "Laxmi Nagar", "Nirman Vihar", "Preet Vihar",
        "Karkarduma", "Anand Vihar ISBT", "Kaushambi", "Vaishali",
    ],
    "Green Line": [
        "Inderlok", "Kirti Nagar", "Ashok Park Main", "Punjabi Bagh",
        "Punjabi Bagh West", "Shivaji Park", "Madipur", "Paschim Vihar East",
        "Paschim Vihar West", "Peera Garhi", "Udyog Nagar",
        "Maharaja Surajmal Stadium", "Nangloi", "Nangloi Railway station",
        "Rajdhani Park", "Mundka", "Mundka Industrial Area", "Ghevra",
        "Tikri Kalan", "Tikri Border", "Pandit Shree Ram Sharma",
        "Bahadurgarh City", "Brigadier Hoshiyar Singh",
    ],
    "Green Line branch": [
        "Ashok Park Main", "Satguru Ramsingh Marg", "Kirti Nagar",
    ],
    "Violet Line": [
        "Kashmere Gate", "Lal Qila", "Jama Masjid", "Delhi Gate", "ITO",
        "Mandi House", "Janpath", "Central Secretariat", "Khan Market",
        "Jawaharlal Nehru Stadium", "Jangpura", "Lajpat Nagar", "Moolchand",
        "Kailash Colony", "Nehru Place", "Kalkaji Mandir", "Govind Puri",
        "Harkesh Nagar Okhla", "Jasola Apollo", "Sarita Vihar",
        "Mohan Estate", "Tughlakabad Station", "Badarpur Border", "Sarai",
        "NHPC Chowk", "Mewala Maharajpur", "Sector 28", "Badkhal Mor",
        "Old Faridabad", "Neelam Chowk Ajronda", "Bata Chowk",
        "Escorts Mujesar", "Sant Surdas (Sihi)", "Raja Nahar Singh",
    ],
    "Pink Line": [
        "Majlis Park", "Azadpur", "Shalimar Bagh", "Netaji Subhash Place",
        "Shakurpur", "Punjabi Bagh West", "ESI Hospital", "Rajouri Garden",
        "Mayapuri", "Naraina Vihar", "Delhi Cantonment",
        "Durgabai Deshmukh South Campus", "Sir Vishweshwaraiah Moti Bagh",
        "Bhikaji Cama Place", "Sarojini Nagar", "INA", "South",
        "Lajpat Nagar", "Vinobapuri", "Ashram", "Hazrat Nizamuddin",
        "Mayur Vihar -I", "Mayur Vihar Pocket I", "Trilokpuri Sanjay Lake",
        "East Vinod Nagar - Mayur Vihar-II", "Mandawali - West Vinod Nagar",
        "IP Extension", "Anand Vihar ISBT", "Karkarduma",
        "Karkarduma Court", "Krishna Nagar", "East Azad Nagar", "Welcome",
        "Jaffrabad", "Maujpur-Babarpur", "Gokulpuri", "Johari Enclave",
        "Shiv Vihar",
    ],
    "Magenta Line": [
        "Janakpuri West", "Dabri Mor-Janakpuri South", "Dashrath Puri",
        "Palam", "Sadar Bazaar Cantonment", "Terminal 1-IGI Airport",
        "Shankar Vihar", "Vasant Vihar", "Munirka", "R.K.Puram",
        "Hauz Khas", "Panchsheel Park", "Chirag Delhi", "Greater Kailash",
        "Nehru Enclave", "Kalkaji Mandir", "Okhla NSIC", "Sukhdev Vihar",
        "Jamia Millia Islamia", "Okhla Vihar", "Jasola Vihar Shaheen Bagh",
        "Kalindi Kunj", "Okhla Bird Sanctuary", "Botanical Garden",
    ],
    "Grey Line": ["Dwarka", "Nangli", "Najafgarh"],
    "Orange Line": [
        "New Delhi", "Shivaji Stadium", "Dhaula Kuan", "Delhi Aerocity",
        "Indira Gandhi International Airport", "Dwarka Sector 21",
    ],
}

# station name (as it appears above) -> set of line keys it interchanges between.
# Only stations genuinely shared by 2+ of the LINES above are listed --
```

```
# Sikandarpur/Rapid Metro Gurgaon and Noida Sector 52/Aqua Line are real DMRC
# interchanges too, but those other networks aren't in this project's data.
INTERCHANGES = {
    "Kashmere Gate": {"Red Line", "Yellow Line", "Violet Line"},
    "Rajiv Chowk": {"Yellow Line", "Blue Line"},
    "Inderlok": {"Red Line", "Green Line"},
    "New Delhi": {"Yellow Line", "Orange Line"},
    "Central Secretariat": {"Yellow Line", "Violet Line"},
    "Dwarka Sector 21": {"Blue Line", "Orange Line"},
    "Kirti Nagar": {"Blue Line", "Green Line", "Green Line branch"},
    "Yamuna Bank": {"Blue Line", "Blue Line branch"},
    "Ashok Park Main": {"Green Line", "Green Line branch"},
    "Mandi House": {"Blue Line", "Violet Line"},
    "Azadpur": {"Yellow Line", "Pink Line"},
    "Netaji Subhash Place": {"Red Line", "Pink Line"},
    "Rajouri Garden": {"Blue Line", "Pink Line"},
    "INA": {"Yellow Line", "Pink Line"},
    "Lajpat Nagar": {"Violet Line", "Pink Line"},
    "Janakpuri West": {"Blue Line", "Magenta Line"},
    "Hauz Khas": {"Yellow Line", "Magenta Line"},
    "Kalkaji Mandir": {"Violet Line", "Magenta Line"},
    "Botanical Garden": {"Blue Line", "Magenta Line"},
    "Anand Vihar ISBT": {"Blue Line branch", "Pink Line"},
    "Karkarduma": {"Blue Line branch", "Pink Line"},
    "Welcome": {"Red Line", "Pink Line"},
    "Mayur Vihar -I": {"Blue Line", "Pink Line"},
    "Dwarka": {"Blue Line", "Grey Line"},
    "Punjabi Bagh West": {"Green Line", "Pink Line"},
}

# Average interchange (platform-to-platform transfer) time in minutes.
# Real DMRC transfers range ~3-7 min depending on the station; Rajiv Chowk
# and Kashmere Gate run long. This is a network-wide average, not per-station.
INTERCHANGE_PENALTY_MIN = 5.0

# Fallback inter-station run+dwell time (minutes) used only when we can't
# compute one from coordinates (missing/placeholder lat-long -- see
# travel_graph.py). DMRC's scheduled average is close to this.
DEFAULT_SEGMENT_MIN = 2.5

# Manual alias map: topology name -> canonical name as it appears in
# DELHI_METRO_DATA.csv, for the handful that don't match after normalization.
ALIASES = {
    "rohini sector 18": "Rohini Sector 18",
    "vishwa vidyalaya": "Vishwa Vidyalaya",
    "barakhambha road": "Barakhambha Road",
    "supreme court": "Supreme Court",
    "sant surdas (sihi)": "Sant Surdas (Sihi)",
    "east vinod nagar - mayur vihar-ii": "East Vinod Nagar - Mayur Vihar-II",
    "mandawali - west vinod nagar": "Mandawali - West Vinod Nagar",
    "r.k.puram": "R.K.Puram",
    "indira gandhi international airport": "Indira Gandhi International Airport",
}
```

Complete topology-aware travel graph and Dijkstra module `app/travel_graph.py:1-167`

```python
"""
Line-topology-aware travel time graph, used by routes.py in place of the old
dmrc_all_station_travel_times.csv (straight-line haversine between every
pair of stations, ~48.5k rows for permutations that mostly aren't reachable
without changing lines).

See network_topology.py for what data was missing and why the old approach
had no alternative but straight-line distance.

Node = (station, line) e.g. ("Rajiv Chowk", "Yellow Line"). This
"line-expanded" graph is what lets us charge an interchange penalty only
when a route actually switches platforms/lines, instead of moving for free
between lines that happen to share a station name.
"""
import math
import heapq
import csv
from collections import defaultdict

from app.network_topology import (
    LINES, INTERCHANGES, INTERCHANGE_PENALTY_MIN, DEFAULT_SEGMENT_MIN, ALIASES,
)

AVG_SPEED_KMH = 32.0  # DMRC's published average commercial speed (incl. dwell)
MAX_SEGMENT_MIN = 8.0  # ~4.3 km -- beyond this between *adjacent* stations, the coordinate is almost certainly wrong, not the track
_STRIP_CHARS = "\u2020\u2021*#\u00a4"


def _normalize(name):
    name = "".join(ch for ch in name if ch not in _STRIP_CHARS)
    return (
        name.strip().lower()
        .replace("-", "-").replace("-", "-").replace("-", "-")
        .replace(",", "").replace(".", "").replace("  ", " ")
    )


def _load_stations(csv_path):
    """Returns {normalized_name: (canonical_name, lat, lon)}."""
    stations = {}
    with open(csv_path, newline="", encoding="utf-8-sig") as f:
        for row in csv.DictReader(f):
            name = "".join(ch for ch in row["Station"].strip() if ch not in _STRIP_CHARS).strip()
            lat = row["Latitude"].strip()
            lon = row["Longitude"].strip()
            lat = float(lat) if lat else None
            lon = float(lon) if lon else None
            stations[_normalize(name)] = (name, lat, lon)
    return stations


def _is_placeholder_coord(lat, lon, coord_counts):
    """Flags coordinates that are obviously a stuck/default value rather than
    a real geocode -- the exact same (lat, lon) recurring across more than 2
    stations (e.g. every Noida Sector X station sharing one point)."""
    if lat is None or lon is None:
        return True
    return coord_counts[(lat, lon)] > 2


def _haversine_km(lat1, lon1, lat2, lon2):
    R = 6371.0
    dlat = math.radians(lat2 - lat1)
    dlon = math.radians(lon2 - lon1)
    a = (math.sin(dlat / 2) ** 2
         + math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) * math.sin(dlon / 2) ** 2)
    return R * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))


def _match_topology_to_canonical(stations):
    mapping = {}
    all_topology_names = set()
    for stops in LINES.values():
        all_topology_names.update(stops)
    all_topology_names.update(INTERCHANGES.keys())

    for name in all_topology_names:
        norm = _normalize(name)
        if norm in stations:
            mapping[name] = stations[norm][0]
            continue
        alias = ALIASES.get(norm)
        if alias and _normalize(alias) in stations:
            mapping[name] = stations[_normalize(alias)][0]
    return mapping


def build_graph(csv_path):
    """Builds the line-expanded graph: {(station, line): {(station, line): minutes}}."""
    stations = _load_stations(csv_path)
    mapping = _match_topology_to_canonical(stations)

    coord_counts = defaultdict(int)
    for _, lat, lon in stations.values():
        if lat is not None and lon is not None:
            coord_counts[(lat, lon)] += 1

    graph = defaultdict(dict)

    for line, stops in LINES.items():
        canon_stops = [mapping.get(s) for s in stops]
        for a, b in zip(canon_stops, canon_stops[1:]):
            if a is None or b is None:
                continue
            _, lat1, lon1 = stations[_normalize(a)]
            _, lat2, lon2 = stations[_normalize(b)]
            if not _is_placeholder_coord(lat1, lon1, coord_counts) and \
               not _is_placeholder_coord(lat2, lon2, coord_counts):
                weight = max((_haversine_km(lat1, lon1, lat2, lon2) / AVG_SPEED_KMH) * 60, 1.0)
                if weight > MAX_SEGMENT_MIN:
                    # Real adjacent DMRC stations are essentially never this
                    # far apart (even the longest suburban stretches run
                    # ~3-4 km). A haversine result this large means the
                    # coordinate itself is wrong -- not a duplicate
                    # placeholder (already caught above), just a bad,
```

```
                    # unique-looking value -- e.g. "Hindon" in the source
                    # data sits ~22 km north of its actual neighbours
                    # Shaheed Sthal and Arthala. Fall back rather than trust it.
                    weight = DEFAULT_SEGMENT_MIN
            else:
                weight = DEFAULT_SEGMENT_MIN
            graph[(a, line)][(b, line)] = weight
            graph[(b, line)][(a, line)] = weight

    for station, lines_here in INTERCHANGES.items():
        canon = mapping.get(station)
        if canon is None:
            continue
        line_list = list(lines_here)
        for i in range(len(line_list)):
            for j in range(i + 1, len(line_list)):
                na, nb = (canon, line_list[i]), (canon, line_list[j])
                graph[na][nb] = INTERCHANGE_PENALTY_MIN
                graph[nb][na] = INTERCHANGE_PENALTY_MIN

    return graph


def shortest_times_from(graph, station_name):
    """Multi-source Dijkstra from every line-node of `station_name` (a station
    with two lines starts the search from both). Returns
    {destination_station_name: minutes}, using whichever line gets there
    fastest and automatically paying the interchange penalty when a route
    needs to switch lines."""
    start_nodes = [n for n in graph if n[0] == station_name]
    dist = {n: 0.0 for n in start_nodes}
    pq = [(0.0, n) for n in start_nodes]
    heapq.heapify(pq)
    visited = set()

    while pq:
        d, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)
        for v, w in graph[u].items():
            nd = d + w
            if nd < dist.get(v, float("inf")):
                dist[v] = nd
                heapq.heappush(pq, (nd, v))

    best_per_station = {}
    for (station, _line), d in dist.items():
        if station not in best_per_station or d < best_per_station[station]:
            best_per_station[station] = d
    best_per_station.pop(station_name, None)
    return best_per_station
```